

Two quietly-useful GitHub Actions updates

Service container entrypoint / command overrides · deployment: false environments

~25 min walkthrough

live demo

Q&A at the end

Agenda

- **0–3 min** · the two features in one breath
- **3–13 min** · Demo 1: service container entrypoint + command
- **13–23 min** · Demo 2: deployment: false environments
- **23–25 min** · webhook harness recap & gotchas
- **25–35 min** · Q&A

The TL;DR

Service container overrides

You can finally set `entrypoint:` and `command:` on a `services:` entry. No more baking custom images just to flip a flag.

shipped early April 2026

`deployment: false`

Use an environment for its *secrets* and *approvals* without producing a `Deployment` object, webhook, or timeline entry.

shipped Q1 2026, GA April

Demo 1 · The pain we're removing

- The `services:` block historically only let you tweak the image with `options:`, which forwards to `docker create`.
- Anything that changed how the *process* ran (auth, init scripts, alt configs) required a custom image push or a step-level hack.
- Result: a fleet of one-flag-different forks of `redis:alpine`, `postgres:16`, etc.

Show of hands: who maintains a custom service image right now?

Demo 1 · Before & after

Before

```
services:
  redis:
    image: redis:7-alpine
    options: >-
      --health-cmd "redis-cli ping"
# password? bake a custom image
# or run CONFIG SET in a step
```

After (April 2026)

```
services:
  redis:
    image: redis:7-alpine
    entrypoint: redis-server

    command: >-
      --requirepass ${ secrets.PW }
      --maxmemory 64mb

    options: >-
      --health-cmd "redis-cli -a $PW ping"
```

Demo 1 • Live

- Trigger `demo-1 service container overrides` via *Run workflow*.
- Three jobs run in parallel:
 - `before-overrides` — baseline, no auth.
 - `with-entrypoint-override` — redis with `--requirepass`.
 - `postgres-bootstrap` — swap `entrypoint` to drop a seed file before `docker-entrypoint.sh`.
- Watch the harness window: each job posts a status payload when it finishes.

Demo 1 · Where this earns its keep

- **Auth-required test fixtures.** Redis, MySQL, Mongo — turn auth on with a flag instead of a Dockerfile.
- **Init-time bootstrap.** Drop seed SQL/JSON into the image's init.d before the real entrypoint runs.
- **Alternate binaries.** Same image, different daemon (e.g. `redis-sentinel` instead of `redis-server`).

Gotcha: entrypoint replaces the image default. If you only need to pass args, set `command:` alone.

Demo 2 · What environments were missing

- An environment: block bundles three things together:
 1. required reviewers / wait timers / branch policies
 2. environment-scoped *secrets* and *variables*
 3. a Deployment + DeploymentStatus record (UI row + webhook)
- Sometimes you want #1 and #2 but not #3.
- Old workaround: a second environment named prod-ops with the same secrets, or worse, repo-level secrets gated by manual review.

Demo 2 • The whole feature in 5 lines

```
jobs:
  audit-prod-config:
    runs-on: ubuntu-latest
    environment:
      name: production
      deployment: false # keep the gate, drop the deployment record

  steps:
    - run: ./scripts/audit.sh
      env:
        API_TOKEN: ${{ secrets.API_TOKEN }}
```

- Required reviewers? still apply
- Environment secrets? still scoped
- deployment webhook? not fired
- Environments UI row? not created

Demo 2 · Live A/B

- Run demo-2 with `target=staging` → ship-it creates a Deployment.
- Run demo-2 with `target=ops-readonly` → audit-prod-config uses the same secrets & reviewers, but emits *no* deployment.
- Switch to the harness UI and filter by `type = deployment`: only one run appears.
- Open the SHA in the Code tab — only one green bubble next to it.

Demo 2 · Where this earns its keep

- **Maintenance workflows** — cache/secret rotation, key rolls, cron-style cleanups.
- **Read-only prod audits** — drift detection, config dumps, compliance scans.
- **Smoke tests post-release** — want prod creds + approval, but the deploy already happened upstream.
- **Data exports** — touching prod data with reviewer approval, not "shipping" anything.

The webhook harness

- `POST /events` — workflow jobs push status here.
- `GET /stream` — Server-Sent Events feed consumed by the dashboard.
- `GET /` — live table; filter by demo, run, or job result.
- Auth: `x-demo-token` header matches `WEBHOOK_TOKEN`.
- Run it as `node webhook-harness/server.js` behind `cloudflared/ngrok`; point the repo variable `WEBHOOK_URL` at the tunnel.

Gotchas

- **Service overrides:** `entrypoint` *replaces* the image default. To just add args, set `command:` only.
- **Service overrides:** shell expansion happens in the runner's shell, not the container — quote carefully.
- **deployment: false:** the environment must still exist in repo settings; approvals/wait-timers still apply.
- **deployment: false:** jobs that previously listened for `deployment_status` webhooks won't see these runs — intentional, but worth auditing.

10 GitHub Actions practices worth knowing

Q&A format · five slides · two practices per slide

1. Scope permissions per job
2. Cancel stale PR runs
3. Pin actions to a commit SHA
4. Cache dependency installs
5. Set explicit job timeouts
6. Reusable workflows over copy-paste
7. Filter triggers with paths
8. Surface test results as annotations
9. Manage artifact retention
10. Scope secrets to the step, not the job

Best practices 1 & 2

Q How do I lock down what a workflow can write to my repo? ^{#1}

A Declare permissions: at the **job level**, not the workflow level. Set contents: read as the workflow default and grant elevated scopes (e.g. packages: write) only to the specific job that needs them. Least-privilege at the job boundary.

```
permissions: read-all    # workflow default
jobs:
  publish:
    permissions:
      packages: write    # escalate only here
```

Q How do I stop wasting CI minutes when developers push rapidly? ^{#2}

A Add a concurrency: block keyed to the ref. Setting cancel-in-progress: true automatically cancels the previous run when a new commit lands on the same branch — only the latest run completes.

```
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

Best practices 3 & 4

Q Is it safe to reference a third-party action with `@v4`? #3

A No — tags are mutable. Pin to a **full commit SHA** for any action you don't control. Add the human-readable tag as a comment. Let Renovate or Dependabot keep the SHA current.

```
# risky
- uses: actions/checkout@v4
# safe
- uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
  # v4.2.2
```

Q My workflow spends 3 minutes on `pip install` every single run. How do I fix it? #4

A Enable the cache: input on `actions/setup-python` (and its siblings). For custom toolchains use `actions/cache` with a key that hashes the lock file — that ensures a changed `requirements.txt` busts the cache automatically.

```
- uses: actions/setup-python@v5
  with:
    python-version: "3.12"
    cache: pip      # key includes requirements hash
```


Best practices 5 & 6

Q A runaway job consumed 6 hours of CI minutes. How do I prevent that? ^{#5}

A Set `timeout-minutes`: on every job. The default is 360 min (6 hours). Most jobs should be < 30 min. Also set it on individual steps for commands that can hang (e.g. integration-test suites).

```
jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 20 # not 360
```

Q I copy-paste the same 50 lines of YAML into every repo. Is there a better way? ^{#6}

A Yes — **reusable workflows** (on: `workflow_call`). Extract the shared logic into a central repo and call it with `uses: .`. Inputs and secrets are passed explicitly; the pattern is one file to patch instead of N.

```
# caller
jobs:
  build:
    uses: org/platform/.github/workflows/build.yml@main
    with:
      node-version: "22"
```

Best practices 7 & 8

Q My workflow triggers on every push, even for docs-only changes. How do I filter? ^{#7}

A Use `on.push.paths:` (and `paths-ignore:`) to restrict triggers to files that affect the build. Combine with `on.pull_request.paths:` for the same effect on PRs. Glob patterns work.

```
on:
  push:
    paths:
      - "src/**"
      - "requirements*.txt"
    paths-ignore:
      - "docs/**"
      - "*.md"
```

Q Our test failures are buried in 2,000 lines of logs. How do we surface them? ^{#8}

A Emit a JUnit XML report and upload it; pair with a reporting action or the job-summary API. Failures appear as **PR annotations** and a structured summary — reviewers see the failing test name, not a wall of text.

```
- run: pytest --junit-xml=report.xml
- uses: dorny/test-reporter@v1
  if: always()
  with:
    reporter: java-junit
    path: report.xml
```

Best practices 9 & 10

Q Artifacts are eating our storage quota. What's the fix? #9

A Set `retention-days`: on every `upload-artifact` step to the minimum useful window (often 3–7 days for build artifacts, 30 for release binaries). Also: only upload what a downstream job or a human will actually read.

```
- uses: actions/upload-artifact@v4
  with:
    name: test-report
    path: report.xml
    retention-days: 7    # not 90
```

Q How do I use a secret in a script without accidentally leaking it into the log? #10

A Scope `env`: to the **step** that needs it, not the job. Pass secrets as environment variables — never interpolate `${{ secrets.X }}` directly into a shell string. The runner masks `env-var` values; inline interpolation bypasses that protection.

```
# risky – interpolated into shell string
- run: curl -H "Authorization: ${{ secrets.TOKEN }}"

# safe – passed as env var, runner masks it
- run: curl -H "Authorization: $API_TOKEN"
  env:
    API_TOKEN: ${{ secrets.TOKEN }}
```

Recap & questions

- Two features, one theme: *fewer workarounds*.
- `services`: — override `entrypoint` + `command`; stop maintaining custom images for one-flag changes.
- `deployment`: `false` — use environments for gates & secrets without polluting the deployments timeline.

repo: github.com/seandorsett/curly-engine · branch: `claude/github-actions-features-71S3r`